

Think of the validation set as a Practice Exam that happens while the model is still studying. To understand why we need it, let's look at the three "piles" of data using a student analogy:

1. The Three Datasets Analogy

- **Training Set (The Textbook):** The student (the model) reads these chapters over and over. They see the questions and the answers at the back of the book. They learn the patterns.
- **Validation Set (The Practice Quiz):** Once a week, the teacher gives the student a quiz. The student hasn't seen these specific questions before.
 - If the student gets a 100% on the textbook but a 40% on the quiz, the teacher knows the student is just **memorizing** (overfitting) instead of **learning** the concepts.
 - The teacher then adjusts the "study strategy" (Hyperparameters like learning rate or number of layers) based on these quiz scores.
- **Test Set (The Final Exam):** This is the high-stakes exam at the end of the semester. It is only taken **once**. It tells you how the student will perform in the real world.

2. Why can't we just use the Test Set for everything?

This is the most common mistake in Data Science.

If you use your **Test Set** to decide which model architecture is better or to tune your settings, the model's "knowledge" starts to leak. You are essentially "peeking" at the final exam to decide how to study.

Eventually, you'll end up with a model that performs perfectly on that specific test set but fails miserably when you deploy it to real customers (like a "Wally" user in a store) because the model was optimized for the exam, not for the subject.

3. What does it look like in practice?

When you are coding your training loop in PyTorch, you will see the "Loss" (error rate) for two different groups at the end of every round (epoch):

Metric	High Training Accuracy / High Validation Accuracy	High Training Accuracy / Low Validation Accuracy
Meaning	Learning: The model is generalizing well.	Overfitting: The model is just memorizing noise.
Action	Keep going!	Stop training early or simplify the model.

Summary

The validation set acts as a **"Neutral Third Party"** during training. It provides a reality check to ensure that the "intelligence" your neural network is developing is actually useful for data it hasn't seen before.

Since you're looking at the MNIST dataset, if your model gets 99% accuracy on the training images but only 70% on the validation images, it means your network is likely too complex and is just "remembering" the specific squiggles of the training set rather than learning what a "7" actually looks like.

Q- so this validation dataset is like a test but after training, and its something similar in ML we use cross validation, which takes different samples of the training dataset and train the model on that. I think cross validation is better than just checking the model performance after training?

Validation (CV) is statistically "better" because it's more robust, but there is a specific reason why we use a single Validation Set for Neural Networks instead.

1. The Direct Correction: Validation is *During*, not *After*

A small but important distinction: In Deep Learning, the validation check happens during the training process (usually at the end of every "epoch" or round).

- ML (Cross-Validation): You train 5 different models on 5 different folds, then look at the average.
 - DL (Validation Set): You train one model, and every 10 minutes, you pause to see how it's doing on the validation set. If the validation score starts getting worse while the training score gets better, you "Early Stop" the training to prevent overfitting.
-

2. Why don't we always use Cross-Validation in Deep Learning?

In traditional Machine Learning (like Linear Regression or Random Forests), we love Cross-Validation. But in Deep Learning (PyTorch), we usually stick to a single Validation Set for three main reasons:

A. The "Time and Money" Problem

Neural networks are "heavy."

- Traditional ML: Training a model might take 5 seconds. Doing 5-fold Cross-Validation takes 25 seconds. No big deal.
- Deep Learning: Training a large model (like a transformer or a high-res image classifier) might take 10 hours. Doing 5-fold CV would take 50 hours. In a business setting like Walmart, that's a massive waste of GPU compute costs.

B. Dataset Size

Cross-Validation is most useful when you have a small dataset (e.g., 500 rows). You need to squeeze every bit of info out of those 500 rows. With a dataset like MNIST (70,000 images), the data is so "dense" that a single random 10% slice is usually a very accurate representation of the whole. The "extra" accuracy you get from CV isn't worth the extra days of training.

C. The Moving Target (Internal State)

Because Neural Networks learn incrementally (step-by-step), we need a fixed "yardstick" to watch the model improve in real-time. If the validation data kept changing (as it does in CV), it would be much harder to tell if the model is actually getting better or if it just got an "easy" fold.

3. Comparison Table

Feature	Single Validation Set (Common in DL)	Cross-Validation (Common in ML)
Speed	Fast (Train 1 model).	Slow (Train K models).
Data Usage	"Wastes" a slice of data (model never sees it).	Uses every single row for both training and testing.
Best For	Large datasets (Big Data).	Small to Medium datasets.
Purpose	To know when to stop training.	To get a highly stable accuracy estimate.

When should YOU use Cross-Validation?

If you are working on your PhD research or a high-stakes project at HEC Montréal where you have a **small, precious dataset** (say, 200 participants), then **Cross-Validation is definitely the way to go**. It ensures your results aren't just a "lucky fluke" of how you split your data.

But for your MNIST project or building "Wally" at Walmart—where you have thousands or millions of rows—a single, solid Validation Set is the industry standard.